



24. Python: Iterators and Generators

Produce values only when needed

Previous Section:

[`</>` 23. Python: Classes and Objects](#)

Contents:

[1. Python Iterators](#)

[Iterable Objects](#)

[The `iter\(\)` Method](#)

[The `next\(\)` Method](#)

[Iterators Maintain Their Own State](#)

[Converting an Iterator Back Into a List](#)

[Lazy Evaluation](#)

[What Makes This Different From a Normal Loop?](#)

[3. Python Generators](#)

[Leverage Lazy Evaluation](#)

[The `yield` Method](#)

[Normal Function vs Generator](#)

[Generators Remember Their State](#)

[A Little Catch: `def main\(\)` vs `if \_\_name\_\_ == "\_\_main\_\_"`](#)

[Other Generator Examples](#)

[Advanced Examples With Classes](#)

[Summary](#)

References

Sometimes, when we work with data in Python, we don't actually want everything all at once.

Think about it. What if we are processing millions of numbers, huge files, endless streams of data, live sensor readings, AI training batches

Loading everything into memory immediately would be expensive, and sometimes impossible. So Python introduces a different philosophy:

■ ***"Only produce data when someone asks for it."***

And this is where **Iterators** and **Generators** come in. Instead of storing everything at once, they produce values one step at a time. This concept is called **Lazy Evaluation**.

And honestly, this idea is everywhere in modern programming: Machine Learning pipelines; Data streaming; File processing; Game loops; APIs; Infinite sequences

At first, iterators may look strange because loops already exist. But behind every `for` loop in Python, there is actually an iterator working silently underneath.

So in this section, we explore:

- What iterators really are
- How Python moves through data one item at a time
- Why `iter()` and `next()` exist
- And later, how generators simplify the entire process

Because once you understand iterators, you start understanding how Python itself thinks.

1. Python Iterators

An **iterator** is an object that allows us to traverse through data one item at a time. Instead of returning everything immediately, the iterator remembers its current position and only produces the next value when requested. This is one of the foundations of Python iteration.

Iterable Objects

Before understanding iterators, we first need to understand something called an **iterable**. An iterable is simply an object that can be looped through.

Examples include: Lists; Tuples; Strings; Dictionaries; Sets

```
numbers = [1, 2, 3, 4]
```

This list is iterable because Python can move through its contents one by one.

The `iter()` Method

To transform an iterable into an iterator, we use: `iter()`. Example:

```
# instantiate a list object
list_instance = [1, 2, 3, 4]

# convert the list to an iterator
iterator = iter(list_instance)

print(iterator)
```

```
<list_iterator object at 0x7d51dc20a9e0>
```

Now `iterator` becomes an iterator object. It remembers where it currently is while traversing the data.

The `next()` Method

To retrieve values manually from the iterator, we use: `next()`. Every call moves the iterator forward by one step. Example:

```
# instantiate a list object
list_instance = [1, 2, 3, 4]

# convert the list to an iterator
iterator = iter(list_instance)

# return items one at a time
print(next(iterator))
print(next(iterator))
print(next(iterator))
print(next(iterator))
```

1
2
3
4

Notice what happens here. The iterator does not give us all values at once. Instead:

- first call → returns 1
- second call → returns 2
- third call → returns 3
- and so on.

The iterator remembers its current position internally.

Iterators Maintain Their Own State

One important detail, different iterators created from the same iterable maintain different positions. Example:

```
list_instance = [1, 2, 3, 4]

iterator_a = iter(list_instance)
iterator_b = iter(list_instance)

print(f"A: {next(iterator_a)}")
print(f"A: {next(iterator_a)}")
print(f"A: {next(iterator_a)}")
print(f"A: {next(iterator_a)}")

print(f"B: {next(iterator_b)}")
```

```
A: 1
A: 2
A: 3
A: 4
B: 1
```

Even though both iterators came from the same list:

- `iterator_a` already moved to the end
- `iterator_b` is still at the beginning

Each iterator tracks its own traversal state independently.

Converting an Iterator Back Into a List

An iterator can also be converted back into a list. Example:

```
list_instance = [1, 2, 3, 4]

# produce an iterator from an iterable
iterator = iter(list_instance)
print(list(iterator))
```

```
[1, 2, 3, 4]
```

But notice something important. Once the iterator is fully consumed, it becomes empty. Example:

```
list_instance = [1, 2, 3, 4]

iterator = iter(list_instance)

print(list(iterator))
print(list(iterator))
```

```
[1, 2, 3, 4]
[]
```

The iterator already reached the end during the first conversion. So the second time, nothing remains.

Other example:

```
list_instance = [1, 2, 3, 4]

iterator = iter(list_instance)
iterator1 = iter(list_instance[:2])

print(list(iterator))
print(list(iterator1))

print(list(iterator))
print(list(iterator1))
```

```
[1, 2, 3, 4]
[1, 2]
[]
[]
```

Lazy Evaluation

This behavior is part of something called **Lazy Evaluation**. Instead of generating all values immediately. Values are produced only when needed, memory usage becomes more efficient, large datasets become easier to process

This is extremely important in real-world systems. Imagine trying to load a 100GB file, an infinite stream, live network packets all at once. Not practical. Iterators solve this by producing values gradually.

What Makes This Different From a Normal Loop?

At first glance, iterators may seem unnecessary because loops already exist. Example:

```
numbers = [1, 2, 3, 4]

for number in numbers:
    print(number)
```

1
2
3
4

Looks much simpler, right? But here's the important secret: **A `for` loop is already using an iterator internally.**

Python silently does something similar to this:

```
iterator = iter(numbers)

while True:
    try:
        item = next(iterator)
        print(item)
    except StopIteration:
        break
```

1
2
3
4

The loop itself is simply hiding the iterator mechanism from us. So why learn iterators directly? Because sometimes we want manual control, partial traversal, lazy computation, infinite sequences, custom iteration behavior

And for those situations, understanding iterators becomes extremely powerful. They are one of the hidden engines behind Python itself.

3. Python Generators

If iterators are the engine behind lazy evaluation, then generators are the shortcut that makes building them easy.

Because honestly, manually creating iterators with: `__iter__()` or `__next__()` can become repetitive very quickly.

So Python gives us something far more elegant: `yield`. And this single keyword completely changes how a function behaves.

Leverage Lazy Evaluation

A normal function computes everything first, then returns the final result. But generators work differently. They produce values one at a time, only when needed.

This means lower memory usage, better performance for large datasets, support for infinite sequences, streaming-style computation.

Instead of giving all results immediately, generators pause and resume execution whenever requested.

The `yield` Method

The `yield` keyword transforms a normal function into a **generator function**.

The moment Python sees `yield`, the function no longer behaves like a regular function, it becomes a generator object, execution pauses after each yield, state is remembered automatically

Normal Function vs Generator

- Normal Function

```
def factors(n):  
    factor_list = []  
  
    for val in range(1, n + 1):  
        if n % val == 0:  
            factor_list.append(val)  
  
    return factor_list  
  
print(factors(20))
```

[1, 2, 4, 5, 10, 20]

This approach stores everything inside a list before returning.

- Using Generator

```
def factors(n):  
    for val in range(1, n + 1):  
        if n % val == 0:  
            yield val  
  
print(factors(20))
```

<generator object factors at 0x7d51ae49fd80>

Notice something important. The generator does not immediately produce values. Instead, it creates a generator object waiting to be iterated. Example:

```
for factor in factors(20):  
    print(factor, end=" ")
```

1 2 4 5 10 20

The values are generated one at a time only when requested.

Generators Remember Their State

This is one of the magical parts of generators. Every time `yield` runs, execution pauses, internal variables are preserved, the function resumes exactly where it stopped. Example:

```
def yield_multiple_statements():
    yield "This is the first statement"
    yield "This is the second statement"
    yield "This is the third statement"
    yield "This is the last statement. Don't call next again!"

example = yield_multiple_statements()

print(next(example))
print(next(example))
print(next(example))
print(next(example))

# This line causes error
# print(next(example))
```

This is the first statement

This is the second statement

This is the third statement

This is the last statement. Don't call next again!

The final `next()` causes `StopIteration`, because the generator has already finished.

A Little Catch: `def main()` vs `if __name__ == "__main__":`

- As programs grow larger, we often organize code using `def main():`. This acts as the main entry point of the program.
- But Python also has a special condition: `if __name__ == "__main__":`

This checks whether the file is being run directly. Example:

```
def main():
    print("Generator demo")
```

```
if __name__ == "__main__":  
    main()
```

Why does this matter? Because generators are commonly used inside reusable modules and pipelines. This condition prevents code from automatically executing when the file is imported elsewhere.



Small Note: `def main()` and `if __name__ == "__main__"` in Module Imports

The combination of: `def main():` and `if __name__ == "__main__":` is not only useful for organizing code. It is also extremely important when working with **multiple Python files (modules)**. Because without it, importing a file may accidentally execute code immediately.

Example Without Protection - Suppose we have two files: `app.py`

```
print("App file executed")  
  
def greet():  
    print("Hello from app.py")
```

and `console.py`

```
import app  
  
print("Console started")
```

App file executed
Console started

Notice something strange. Just importing `app` already executed the print statement automatically.

This becomes dangerous in larger projects because importing a module might run unwanted code, start programs accidentally, open files unexpectedly, trigger database connections, launch infinite loops

Proper Solution - We protect executable code using: `if __name__ == "__main__":`

```
# app.py
def greet():
    print("Hello from app.py")

def main():
    print("App file executed")

if __name__ == "__main__":
    main()
```

```
# console.py
import app

print("Console started")

app.greet()
```

Console started
Hello from app.py

Now the `main()` function only runs when `app.py` itself is executed directly. But when imported into another file: functions, classes, generators, variables, can still be reused safely without executing the entire script automatically.

This becomes extremely important in modular programming, reusable libraries, APIs, Machine Learning projects, large Python applications. Because most real-world Python programs are split across many files. And this small line: `if __name__ == "__main__":` helps separate reusable code from executable program logic, which is one of the foundations of clean Python project structure.

Other Generator Examples

- Custom Range Generator

```
def custom_range(start, stop, step=1):
    current = start

    while current < stop:
        yield current
        current += step

for num in custom_range(2, 10, 2):
    print(num, end=" ")
```

2 4 6 8

- Vowel Filter

```
def vowel_filter(text):
    vowels = "aeiou"

    for char in text:
        if char.lower() in vowels:
            yield char

for vowel in vowel_filter("Hello, World!"):
    print(vowel, end=" ")
```

e o o

- Fibonacci Generator

```
def fibonacci(n):
    a, b = 0, 1

    for _ in range(n):
        yield a
        a, b = b, a + b

for num in fibonacci(8):
    print(num, end=" ")
```

0 1 1 2 3 5 8 13

- **Infinite Generator:** One of the biggest strengths of generators is that they can produce infinite sequences.

```
def infinite_counter(start=1):
    current = start

    while True:
        yield current
        current += 1

for num in infinite_counter():
    if num > 5:
        break

    print(num, end=" ")
```

1 2 3 4 5

A normal list cannot practically do this. Generators can.

- **Manual Iterator Handling:** Generators internally work with iterators.

```

numbers = [10, 20, 30]

iterator = iter(numbers)

while True:
    try:
        value = next(iterator)
        print(value, end=" ")

    except StopIteration:
        break

```

10 20 30

- **Generator Pipelines:** Generators become extremely powerful when chained together. This is called a **pipeline** → Number Producer + Squarer

```

def number_producer(iterable):
    for item in iterable:
        yield item

def squarer(iterable):
    for num in iterable:
        yield num ** 2

numbers = [1, 2, 3, 4, 5]

pipeline = squarer(number_producer(numbers))

for value in pipeline:
    print(value, end=" ")

```

1 4 9 16 25

Data flows lazily through multiple stages. This idea is used heavily in Machine Learning; Big Data; Streaming systems; Data engineering

- **Flatten Nested Lists**

```
def flatten(nested):  
    for item in nested:  
        if isinstance(item, list):  
            yield from flatten(item)  
        else:  
            yield item  
  
nested = [1, [2, [3, 4], 5], [6, 7], 8]  
  
for value in flatten(nested):  
    print(value, end=" ")
```

1 2 3 4 5 6 7 8

Notice the use of: `yield from` . This delegates yielding to another generator.

- **Batch Processing**

```
def batch(iterable, batch_size):  
    items = list(iterable)  
  
    for i in range(0, len(items), batch_size):  
        yield items[i : i + batch_size]  
  
for chunk in batch(range(1, 11), 3):  
    print(chunk)
```

[1, 2, 3]
[4, 5, 6]
[7, 8, 9]
[10]

- **Prime Number Generator**

```

def prime_sieve(limit):
    is_prime = [True] * (limit + 1)

    is_prime[0] = is_prime[1] = False

    for i in range(2, int(limit ** 0.5) + 1):
        if is_prime[i]:
            is_prime[i * i :: i] = [False] * len(is_prime[i
* i :: i])

    for i in range(2, limit + 1):
        if is_prime[i]:
            yield i

for prime in prime_sieve(30):
    print(prime, end=" ")

```

2 3 5 7 11 13 17 19 23 29

- **Running Average**

```
def running_average(iterable):
    total = 0
    count = 0

    for value in iterable:
        total += value
        count += 1

        yield total / count

numbers = [10, 20, 30, 40, 50]

for avg in running_average(numbers):
    print(avg, end=" ")
```

10.0 15.0 20.0 25.0 30.0

- Sliding Window

```
def sliding_window(sequence, n):
    items = list(sequence)

    for i in range(len(items) - n + 1):
        yield tuple(items[i : i + n])

sequence = [1, 2, 3, 4, 5]

for window in sliding_window(sequence, 3):
    print(window, end=" ")
```

(1, 2, 3) (2, 3, 4) (3, 4, 5)

- Unique Values Generator

```
def unique(iterable):
    seen = set()

    for item in iterable:
        if item not in seen:
            seen.add(item)
            yield item

items = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]

for value in unique(items):
    print(value, end=" ")
```

3 1 4 5 9 2 6

Advanced Examples With Classes

Generators are easier to write, but sometimes we manually create iterators ourselves.

- Even Iterator

```

class EvenIterator:
    def __init__(self, limit):
        self.limit = limit
        self.current = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.current > self.limit:
            raise StopIteration

        value = self.current
        self.current += 2

        return value

for num in EvenIterator(10):
    print(num, end=" ")

```

0 2 4 6 8 10

- Reverse String Iterator

```

class ReverseStringIterator:
    def __init__(self, text):
        self.text = text
        self.index = len(text) - 1

    def __iter__(self):
        return self

    def __next__(self):
        if self.index < 0:
            raise StopIteration

        char = self.text[self.index]
        self.index -= 1

        return char

for char in ReverseStringIterator("hello"):
    print(char, end=" ")

```

o l l e h

- Float Step Iterator

```

class FloatStepIterator:
    def __init__(self, start, stop, step):
        self.current = start
        self.stop = stop
        self.step = step

    def __iter__(self):
        return self

    def __next__(self):
        if self.current >= self.stop:
            raise StopIteration

        value = self.current
        self.current = round(self.current + self.step, 10)

        return value

for num in FloatStepIterator(0.0, 1.0, 0.25):
    print(num, end=" ")

```

0.0 0.25 0.5 0.75

Summary

Iterators and Generators are some of the most abstract concepts in Python. And honestly, many learners—and even experienced programmers—do not fully pay attention to them at first.

Because normal loops already seem enough. But underneath Python itself, iteration is everywhere.

Generators and iterators power loops, pipelines, streaming systems, data processing, asynchronous programming, Machine Learning workflows

We explored:

- iterable objects
- `iter()` and `next()`

- lazy evaluation
- generator functions
- the `yield` keyword
- infinite generators
- pipelines
- custom iterator classes

Most importantly, these concepts teach a very powerful programming philosophy:

"Do not compute everything immediately. Produce values only when needed."

And once you understand that idea, you begin thinking about performance, scalability, and memory efficiency in a completely different way.

References

<https://www.datacamp.com/tutorial/python-iterators-generators-tutorial>

<https://realpython.com/python-iterators-iterables/>

<https://pynative.com/python-iterators-generators-exercises/>

Next Section:

[`</>` 25. Python: Multi-Dimensional Lists](#)